C64uRemote für M5Dial

Technische Dokumentation

Version 1.0

Inhaltsverzeichnis

Überblick **2**

Hardware **2**
- Zielplattform . . . 2
- Interner I²C-Bus . . . 3
- Freie GPIOs . . . 3

microSD nachrüsten **3**
- Zweck . . . 3
- Modulauswahl . . . 3
- Verdrahtung . . . 4
- Versorgungsspannung . . . 4
- Aufbau . . . 4
- Inbetriebnahme . . . 4
- Taktstufen . . . 4

Akkubetrieb und Ausschalten **5**
- Akku anschließen . . . 5
- Akkuspannung . . . 5
- Drei Wege zum Ausschalten . . . 5
- Ruhestrom und Standzeit . . . 6

Programmierumgebung **6**
- PlatformIO (empfohlen) . . . 6
- Konfiguration (build_env.h) . . . 6

WLAN-Subsystem **7**
- Laufzeitkonfiguration statt Compile-Zeit . . . 7
- NVS-Layout . . . 7
- Kartenformat . . . 7
- /wifi.txt auf der SD-Karte . . . 7
- Setup-Portal . . . 8
- Direktmodus . . . 8
  - Netznummer (seit v1.5.0) . . . 9

Softwarearchitektur **9**
- Zustandsmodell . . . 9
- Rendering auf dem runden Display . . . 9

ReST-Anbindung an den C64 Ultimate **10**
- Verwendete Endpunkte . . . 10
- Abgewiesene Verbindungen . . . 11
- Wenn der c64u fehlt . . . 11
- Eigener HTTP-Weg (seit v1.4.0) . . . 11
  - NFC-Funkfeld . . . 12
  - Mesh und Roaming . . . 12
  - Netzdiagnose . . . 12
- Wiederverbinden mit mehreren Netzen . . . 12

CPU-Speed . . . 12
Joystick-Ports . . . 12
Disk-Images und Autostart . . . 13
Streaming-Upload . . . 13

NFC-Subsystem **13**
Zugriff auf den Leser . . . 13
Abfragestrategie . . . 13
Warum eine eigene Probe . . . 14
Rueckkehr zum Startbild . . . 14
Kartentypen . . . 14
Kommandokarten . . . 15
PowerOff mit Bestaetigung . . . 15
Karten beschreiben . . . 15
Datenformat . . . 15
NDEF-Parser: Toleranz . . . 16
NFC-Info . . . 16
Kopieren (Dump/Restore) . . . 16

Bedienlogik **16**
Drehgeber . . . 16
Taste . . . 16
Touch . . . 17
Blockierende Aufrufe und Eingaben . . . 17
PowerOff-Absicherung . . . 17

Persistenz **17**

Projektstruktur **17**

Fehlerdiagnose **18**

Quellen **18**

Lizenz **18**

Entstehung **19**

Überblick

C64uRemote ist eine Firmware für den M5Dial (ESP32-S3), die den Commodore 64 Ultimate bzw. Ultimate64 Elite-II über dessen ReST-API fernsteuert. Zusätzlich nutzt sie den im M5Dial verbauten RFID/NFC-Leser (NXP WS1850S) und eine nachgerüstete microSD-Karte, um Programme per NFC-Karte zu starten und Karten zu verwalten.

Die Firmware ist als einzelne Übersetzungseinheit (`src/main.cpp`, rund 4150 Zeilen) umgesetzt und nutzt das Arduino-Framework für ESP32.

Grundlage: Originalprojekt von Karl Prosser (github.com/ReadyOS-C64/C64uRemote), Portierungen für M5StickC Plus2 und M5Dial von Martin Oswald (1MHz.de). Diese Fassung überträgt den vollen Funktionsumfang der M5Stack-Core-Version auf den M5Dial.

Hardware

Zielplattform

Merkmal	Wert
Board	M5Dial (Modul M5StampS3)
SoC	ESP32-S3, 240 MHz, 8 MB Flash, **kein PSRAM**
Display	240 × 240 rund, GC9A01, über M5GFX

Merkmal	Wert
Bedienung	Drehgeber (GPIO 40/41), Taste (GPIO 42), Touch FT3267
RFID/NFC	WS1850S, fest verbaut
Sonstiges	RTC BM8563, Summer

Interner I²C-Bus

Der M5Dial führt Touchcontroller, RTC und RFID-Leser auf einem internen I²C-Bus zusammen:

Gerät	Adresse	Pins
FT3267 (Touch)	0x38	SDA = GPIO 11, SCL = GPIO 12
BM8563 (RTC)	0x51	SDA = GPIO 11, SCL = GPIO 12
WS1850S (RFID)	0x28	SDA = GPIO 11, SCL = GPIO 12

Für den RFID-Leser ist damit **keine Verkabelung und keine Unit RFID2 nötig** – anders als bei der Core-Version.

Freie GPIOs

Nach außen führt der M5Dial nur zwei Grove-Buchsen (HY2.0-4P):

Buchse	Farbe	Pin 1 (schwarz)	Pin 2 (rot)	Pin 3 (gelb)	Pin 4 (weiß)
Port A	rot	GND	5 V	GPIO 13 (SDA)	GPIO 15 (SCL)
Port B	schwarz	GND	5 V	GPIO 2	GPIO 1

Achtung bei Port B: Die gelbe Ader (Pin 3) ist GPIO 2, die weiße (Pin 4) GPIO 1 – die kleinere Nummer liegt also außen. Bis v1.2.1 stand das in dieser Dokumentation vertauscht.

Das sind genau vier freie Signalpins – exakt so viele, wie SPI für eine SD-Karte braucht. Beide Ports werden deshalb belegt.

microSD nachrüsten

Zweck

Der M5Dial hat ab Werk keinen Speicherkartenslot. Ohne SD-Karte funktionieren Fernsteuerung und Karten-Info; das Starten von Programmen und die Funktionen NFC-Dump und NFC-Restore brauchen jedoch einen Massenspeicher, weil die Programmdateien von dort gelesen werden.

Modulauswahl

Geeignet ist jedes einfache SPI-Breakout mit den Anschlüssen `VCC`, `GND`, `MISO`, `MOSI`, `SCK` und `CS`. Zwei Bauformen sind verbreitet:

Bauform	Merkmal	Versorgung
Breakout ohne Regler, ca. 18,5 × 17,5 mm	nur Kartenhalter und Stiftleiste	**3,3 V**
Modul mit Regler und Pegelwandler, ca. 42 × 24 mm	`AMS1117-3.3` und `74LVC125` bestückt	5 V

Die Unterscheidung ist wichtig: Die Grove-Buchsen des M5Dial liefern **5 V**. Ein Breakout ohne Regler gibt diese Spannung direkt an die Speicherkarte weiter, und Micro-SD-Karten sind ausschließlich für 3,3 V spezifiziert.

Verdrahtung

SD-Modul	M5Dial	Ader	GPIO	Konstante in `main.cpp`
`SCK` / `CLK`	Port A, Pin 4	weiß	15	`kSdSckPin`
`MOSI` / `SI` / `CMD`	Port A, Pin 3	gelb	13	`kSdMosiPin`
`MISO` / `SO` / `DAT0`	Port B, Pin 3	gelb	2	`kSdMisoPin`
`CS` / `SS`	Port B, Pin 4	weiß	1	`kSdCsPin`
`GND`	Port A oder B, Pin 1	schwarz	–	–
`VCC`	siehe unten	rot	–	–

Die Zuordnung lässt sich in vier Zeilen am Kopf von `src/main.cpp` ändern:

```
constexpr int kSdSckPin  = 15;   // Port A, Pin "SCL"
constexpr int kSdMosiPin = 13;   // Port A, Pin "SDA"
constexpr int kSdMisoPin = 2;    // Port B, Pin 3 (gelbe Ader)
constexpr int kSdCsPin   = 1;    // Port B, Pin 4 (weisse Ader)
```

Versorgungsspannung

Modul mit Regler: `VCC` direkt an die 5 V einer Grove-Buchse.

Breakout ohne Regler: `VCC` an 3,3 V. Am unauffälligsten ist ein kleiner LDO (HT7333, MCP1700, AMS1117-3.3) zwischen der 5-V-Leitung der Grove-Buchse und `VCC` des Moduls; ein Abblockkondensator von 1 µF an Ein- und Ausgang genügt. Alternativ 3,3 V direkt am StampS3 abgreifen – das ist ein Eingriff im Gehäuse und nur zu empfehlen, wenn du dort ohnehin lötest.

Die Signalleitungen sind unkritisch: Der ESP32-S3 gibt 3,3 V aus, und ein Breakout ohne Pegelwandler reicht das unverändert an die Karte weiter. Ein Modul mit `74LVC125` erzeugt aus 3,3-V-Eingangspegeln ebenfalls gültige Pegel. Kritisch ist ausschließlich eine 5-V-Versorgung eines Moduls ohne Regler.

Aufbau

1. Ein Grove-Kabel für Port A und eines für Port B ablängen. Kurz halten – die SPI-Leitungen sind unabgeschirmt und laufen mit bis zu 20 MHz.
2. Beide `GND`-Adern auf den `GND`-Anschluss des Moduls zusammenführen.
3. Die 5-V-Ader von Port A auf den Reglereingang (bzw. direkt auf `VCC`, falls das Modul einen Regler hat). Die 5-V-Ader von Port B bleibt unbenutzt und wird isoliert.
4. Signaladern gemäß Tabelle anlöten.
5. Vor dem ersten Einschalten `VCC` gegen `GND` auf Kurzschluss prüfen und die Spannung am Modul messen: 3,2 bis 3,4 V bzw. 4,8 bis 5,1 V.

Inbetriebnahme

Eine mit FAT32 formatierte Karte einlegen und das Gerät starten. Die serielle Ausgabe (115200 Baud) meldet:

```
C64uRemote M5Dial v1.3.1 (2026-09-25)  RFID:1  SD:1  Canvas:1  Heap:…
```

`SD:1` bedeutet erkannt. Im Display steht auf allen Menüseiten unten rechts ein grünes **SD**.

Taktstufen

`initSd()` versucht der Reihe nach 20 MHz, 4 MHz und 1 MHz:

```
app.sdReady = SD.begin(kSdCsPin, sdSpi, 20000000);
if (!app.sdReady) app.sdReady = SD.begin(kSdCsPin, sdSpi, 4000000);
if (!app.sdReady) app.sdReady = SD.begin(kSdCsPin, sdSpi, 1000000);
```

Damit läuft auch ein Aufbau mit längeren Grove-Kabeln, nur eben langsamer. Der SD-Bus liegt auf **SPI3 (HSPI)**; SPI2 gehört dem Display.

Akkubetrieb und Ausschalten

Akku anschließen

Der M5Dial hat eine Akkubuchse (JST 1,25 mm, 2-polig) für eine einzelne Li-Ion-/LiPo-Zelle (3,7 V nominal). Geladen wird über USB-C; die Ladeschaltung sitzt auf der Platine. Einen Ein-/Ausschalter braucht es nicht, denn der M5Dial schaltet sich selbst:

Signal	GPIO	Funktion
HOLD	46	Selbsthaltung. HIGH = bleibt an, LOW = trennt den Akku
WAKE / Taste	42	Einschalten; im Betrieb die normale Taste (LOW = gedrückt)

`M5Dial.begin()` (M5Unified) setzt G46 gleich am Anfang auf HIGH. Laut M5Stack fließen im ausgeschalteten Zustand etwa **1,9 µA** bei 4,2 V.

Akkuspannung

Die Spannung lässt sich **ohne Zusatzhardware nicht anzeigen**. Der M5Dial hat weder einen Spannungsteiler vom Akku auf einen ADC-Pin noch einen auslesbaren Lade-IC (dessen CHRG-/STDBY-Ausgänge sind nicht mit dem ESP32 verbunden). M5Unified kennt für den M5Dial keinen PMIC; `M5.Power.getBatteryLevel()` liefert deshalb **-2**. Die vier freien GPIOs sind zudem von der SD-Karte belegt.

Nachrüsten ließe sich ein Fuel-Gauge-IC wie der **MAX17048** am internen I²C-Bus (G11/G12). Er hat die Adresse **0x36** und kollidiert damit weder mit Touch (0x38), RTC (0x51) noch RFID (0x28).

Drei Wege zum Ausschalten

Langer Druck auf „c64u Power Off". In `handleButton()` wird der Punkt `kMenuPowerOff` im Ring-Menü gesondert behandelt: Die Taste muss `kDialOffHoldMs` (1,5 s) gehalten werden, dann folgt direkt `shutdownDevice()`. Das allgemeine Kurzzeichen nach 600 ms (zurück) entfällt auf diesem Symbol; wird früher losgelassen, läuft wie gewohnt `handleSelect()` mit der c64u-Abfrage. `handleTouch()` macht dasselbe, wenn die Berührung auf dem Symbol begann (`menuIndexFromTouch()`); das Touch-Kurzzeichen wird dort nicht ausgelöst.

Befehlskarte `CMD:M5OFF` (auch `CMD:DIALOFF`): `CardCmd::DialOff` ruft ohne Nachfrage `shutdownDevice()` auf. In den ersten `kDialOffBootGuardMs` (8 s) nach dem Start wird die Karte ignoriert (*KARTE ABNEHMEN*), sonst würde eine aufliegende Karte das Gerät bei jedem Einschalten gleich wieder abschalten. In der Liste von *NFC-Cmd* steht der Befehl als *M5Dial Power Off*.

Einstellungen → „M5Dial Power Off". Ab v1.3.0 steht er direkt unter *WLAN*. Wie *WLAN* wird er in `activateSetting()` vor der NFC-Prüfung behandelt und braucht deshalb keinen NFC-Leser.

1. Erster Druck: `requestShutdown()` merkt sich die Anforderung und zeigt *M5DIAL OFF? NOCHMAL!*; in der Liste steht *NOCHMAL*.
2. Zweiter Druck innerhalb von `kShutdownAskMs` (3 s): `shutdownDevice()`.

`shutdownDevice()` zeigt *AUS*, **wartet, bis die Taste losgelassen ist** (sie überbrückt sonst die Selbsthaltung), beendet SD und WLAN und zieht G46 auf LOW. Im Akkubetrieb ist das Gerät damit sofort aus.

Hängt USB oder eine externe Quelle dran, läuft der ESP32 trotz HOLD = LOW weiter. Nach 1,5 s zeigt die Firmware deshalb kurz *USB: Schlaf*, schaltet das Display ab und geht in den Light-Sleep; geweckt wird per GPIO-Wakeup an G42 (G42 ist kein RTC-Pin, Deep-Sleep-Wecken per ext0 geht dort nicht). Nach dem Wecken folgt `esp_restart()`. HOLD bleibt im Schlaf LOW: Wer das Kabel dann abzieht, hat den M5Dial ganz ausgeschaltet.

Alternativ schaltet auch der **RST-Knopf** auf der Rückseite im Akkubetrieb aus, weil G46 während des Resets nicht mehr getrieben wird.

Ruhestrom und Standzeit

Ausgeschaltet zieht der M5Dial laut Datenblatt rund 2 µA; die Schutzschaltung einer typischen Zelle kommt mit 1 bis 6 µA hinzu. Für einen 650-mAh-Akku sind das zusammen unter 0,1 mAh pro Monat – vernachlässigbar. Entscheidend ist die **Selbstentladung** der Zelle selbst (etwa 1 bis 3 % pro Monat). Voll geladen hält der Akku im ausgeschalteten Zustand also viele Monate; nach einem halben Jahr sollte er noch über 80 % haben.

Voraussetzung: Das SD-Modul hängt an der 5-V-Leitung der Grove-Buchsen und ist damit mit ausgeschaltet. Einmal nachmessen lohnt sich: Multimeter in µA-/mA- Stellung in die Plusleitung des Akkus, M5Dial ausschalten – angezeigt werden sollten wenige µA.

Programmierumgebung

PlatformIO (empfohlen)

Die Konfiguration liegt versioniert in `platformio.ini`, wodurch Builds reproduzierbar sind.

Einstellung	Wert	Begründung
`board`	`m5stack-stamps3`	Der M5Dial ist ein StampS3
`platform`	`espressif32@^6.9.0`	Arduino-Framework für ESP32-S3
`board_build.partitions`	`huge_app.csv`	WiFi + SD + RFID überschreiten die Standard-App-Partition
`build_flags`	`-DARDUINO_USB_CDC_ON_BOOT=1, -DARDUINO_USB_MODE=1`	native USB-CDC für den seriellen Monitor
`monitor_speed`	115200	serielle Ausgabe

Bibliotheken (`lib_deps`):

- `m5stack/M5Dial` (^1.0.2) — Display, Touch, Drehgeber **und** der RFID-Treiber (`MFRC522`)
- `m5stack/M5Unified` (^0.2.8) und `m5stack/M5GFX` (^0.2.11)
- `bblanchon/ArduinoJson` (^6.21.5) — bewusst auf Version 6, da Version 7 `DynamicJsonDocument` nicht mehr kennt

Eine separate MFRC522-Bibliothek wie bei der Core-Version wird **nicht** gebraucht: Die M5Dial-Bibliothek bringt eine Klasse `MFRC522` mit, deren API der dort verwendeten `MFRC522_I2C` entspricht.

Befehle:

```
pio run                 # kompilieren
pio run -t upload       # flashen
pio device monitor      # serielle Ausgabe
```

Kommt kein serieller Port zustande: den StampS3 beim Einstecken mit gedrückter Taste in den Download-Modus bringen, flashen, danach einmal Reset.

Konfiguration (build_env.h)

`build_env.h` liefert nur noch die **Startwerte**. Sobald im NVS eine Netzkonfiguration steht, wird die Datei ignoriert. Vorlage `build_env.h.example` nach `build_env.h` kopieren und ausfüllen:

```
#define C64U_WIFI_SSID       "MeinWLAN"
#define C64U_WIFI_PASSWORD   "MeinPasswort"
#define C64U_TARGET_HOST     "192.168.0.64"
#define C64U_TARGET_PASSWORD ""       // nur falls im C64 gesetzt
```

Fehlt die Datei, kompiliert das Projekt mit leeren Defaults; der M5Dial zeigt beim Start dann *SETTINGS > WLAN* und lässt sich am Gerät einrichten.

WLAN-Subsystem

Laufzeitkonfiguration statt Compile-Zeit

SSID, Passwort, c64u-Adresse und c64u-Passwort liegen zur Laufzeit im NVS und sind über *Settings → WLAN* änderbar. Bis zu `kWifiProfileMax` (= 4) Profile werden vorgehalten:

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax];
size_t      gWifiCount;
size_t      gWifiTry;      // Profil fuer den naechsten Verbindungsversuch
```

`beginWiFi()` nimmt `gWifiProfiles[gWifiTry]` und rückt den Index anschließend um eins weiter. Schlägt eine Verbindung fehl, probiert `serviceWiFi()` nach `kWiFiRetryMs` das nächste Profil – über mehrere Durchläufe wandert der Versuch so durch alle bekannten Netze. Beim Start sucht `wifiPickBestProfile()` einmal die Umgebung ab und startet mit dem stärksten bekannten Netz.

`wifiAddProfile()` sortiert ein neues Netz vorn ein; ein bereits bekanntes Netz bekommt nur ein neues Passwort und rutscht ebenfalls nach vorn. Das älteste Profil fällt bei Bedarf hinten heraus.

NVS-Layout

Namensraum `c64unet`, getrennt von den Bedieneinstellungen in `c64udial`:

Schlüssel	Inhalt
`wn`	Anzahl der Profile (0…4)
`s0`…`s3`	SSID
`p0`…`p3`	Passwort
`host`	Adresse des c64u
`hpass`	Passwort des c64u
`dmode`	Direktmodus an/aus
`dssid`, `dpass`	SSID und Passwort des Direktnetzes
`dnet`	Adressbereich des Direktnetzes, z. B. `192.168.4`

Factory Reset betrifft nur `c64udial`; die Netzkonfiguration bleibt erhalten und wird ausschließlich über *WLAN → Alle löschen* verworfen.

Kartenformat

WLAN-Karten benutzen dasselbe Schema wie WLAN-QR-Codes, gespeichert als gewöhnlicher NDEF-Textrecord:

```
WIFI:S:<ssid>;T:WPA;P:<passwort>;;
```

`parseWifiText()` wertet die Felder `S` und `P` aus, akzeptiert mit Backslash maskierte Sonderzeichen und versteht zusätzlich die Kurzform `WIFI:<ssid>;<passwort>`. `wifiCardText()` ist das Gegenstück und wird von *WLAN → Auf NFC-Karte* benutzt.

Auf der Einrichtungsseite (`ScreenMode::WifiCard`) gilt ein Kartentext ohne `WIFI:`-Präfix als reines Passwort für das vorher gewählte Netz.

Außerhalb der Einrichtung – also auf dem Startbild, im Ring-Menü und auf der Leseseite – erkennt `processCard()` eine vollständige `WIFI:`-Karte, speichert das Netz und verbindet sofort. Dabei wird `gWifiTry` gezielt auf dieses Profil gesetzt und bis zu `kWifiCardConnectMs` (8 s) auf `WL_CONNECTED` gewartet; danach übernimmt wieder `serviceWiFi()`. Ist das Netz bereits verbunden, bricht die Funktion mit *SCHON VERBUNDEN* ab – das verhindert eine Endlosschleife, wenn die Karte liegen bleibt und die Hintergrundabfrage sie erneut erkennt.

/wifi.txt auf der SD-Karte

`loadWifiFromSd()` liest ein einfaches Schlüssel-Wert-Format. Jede neue `ssid`-Zeile beginnt einen Eintrag; `#` und `;` leiten Kommentare ein. Zusätzlich werden `host` und `hostpass` erkannt.

```
ssid = MeinWLAN
pass = geheim

host     = 192.168.0.64
hostpass =
```

Gelesen wird die Datei beim Start (nur solange `gWifiCount == 0`) und auf Anforderung über *WLAN → Von SD laden*.

`saveWifiToSd()` ist das Gegenstück: Es schreibt alle Profile samt `host` und `hostpass` mit einem Kommentarkopf zurück nach `/wifi.txt`. Eine vorhandene Datei wandert vorher per `SD.rename()` nach `/wifi.bak`; schlägt das fehl, wird sie gelöscht. Die Funktion liefert die Anzahl der geschriebenen Netze und setzt bei Fehlern einen Klartexthinweis in `errorOut`.

Setup-Portal

`startPortal()` schaltet auf `WIFI_AP` um, öffnet einen Accesspoint auf festem Kanal 1 (`kPortalSsid` / `kPortalPass`), startet einen `DNSServer` als Captive-Portal-Umleitung und einen `WebServer` mit zwei Routen (`/` und `/save`). Der Station-Teil wird bewusst abgeschaltet: bliebe er aktiv, würde er im Hintergrund weiter nach dem gespeicherten Netz suchen, dabei den Funkkanal wechseln und angemeldete Clients abwerfen.

`servicePortal()` läuft in jeder Schleife mit, hält die Leerlaufuhr an, solange ein Client verbunden ist, und beendet das Portal nach `kPortalIdleMs` (5 min) oder `kPortalCloseMs` nach einem erfolgreichen Speichern. `stopPortal()` stellt `WIFI_STA` wieder her und verbindet sofort neu.

Direktmodus

Für Treffen ohne Router spannt der M5Dial ein eigenes WLAN auf (`WIFI_AP`, ohne Station-Teil – aus demselben Grund wie beim Setup-Portal: eine suchende STA wechselt den Kanal und wirft angemeldete Geräte hinaus).

SSID / Passwort	`C64uRemote-Direct` / `c64ultimate` (Vorgaben, änderbar)
Kanal, max. Clients	6, 8
Adresse M5Dial	`<netz>.1`
DHCP-Bereich	`<netz>.64` … `<netz>.74`
Ziel c64u	`<netz>.64`
`<netz>`	`192.168.4` (Vorgabe) oder `192.168.2`, per `direct_net` beliebig

Reihenfolge beim Start: erst `WiFi.softAP()`, dann `WiFi.softAPConfig(ip, ip, 255.255.255.0, <netz>.64)`. Der vierte Parameter setzt den Anfang des DHCP-Bereichs (Arduino-Core 2.0.17 vergibt ab dort elf Adressen). Andersherum kann je nach Core-Version die Vorgabe `192.168.4.1` stehen bleiben – beim Setup-Portal fällt das nicht auf, weil es genau diese Adresse benutzt.

Zieladresse: `targetHost()` liefert im Direktmodus `gDirectHost` statt der gespeicherten Heimadresse, die dadurch unangetastet bleibt. Ist das Gerät als normaler Client mit einem Netz verbunden, dessen SSID der des Direktnetzes entspricht, gilt ebenfalls `<netz>.64` – so kann ein zweiter Fernbediener mitlaufen, ohne dass seine Einstellungen geändert werden.

c64u finden: Die erste DHCP-Vergabe ist `.64`. War ein anderes Gerät schneller, sucht `directNextCandidate()` weiter: Kandidaten sind `.64` und alle Adressen aus `esp_wifi_ap_get_sta_list()` + `esp_netif_get_sta_list()`. Nach jedem gescheiterten Statustest wird der nächste versucht. Als c64u gilt, wer auf `/v1/version` mit JSON antwortet (oder mit 401/403, wenn ein Passwort fehlt). Hat eine Adresse schon einmal geantwortet, wird erst nach dem zweiten Fehlschlag in Folge gewechselt – der c64u weist gelegentlich von sich aus ab (siehe *Abgewiesene Verbindungen*).

Statustest: Ist niemand angemeldet, entfällt der Test ganz. Kommt ein Gerät hinzu (`WiFi.softAPgetStationNum()` steigt) und ist der c64u noch nicht bestätigt, wird sofort geprüft, wieder beginnend bei `.64`. Weil der c64u dann oft noch mit DHCP beschäftigt ist, folgen

bis zu sechs Nachprüfungen im Abstand von 3 s (`directRecheckDue()`), auf jeder Seite, bis er antwortet. Im Direktnetz gilt für den Verbindungsaufbau außerdem eine Frist von 1 s.

DHCP zurücksetzen: Der DHCP-Server des ESP32 vergibt Adressen über einen Zeiger, der nur vorwärts läuft. Gibt der c64u beim Abmelden seine Adresse frei oder fragt er beim Wiederkommen nach der alten, verliert der Server den Eintrag und vergibt die nächste (`.65`, `.66` …). Deshalb startet `directRestartDhcp()` den Server neu, sobald sich das letzte Gerät abgemeldet hat – danach beginnt er wieder bei `.64`. Mit weiteren angemeldeten Geräten unterbleibt das, weil der Server dabei alle Zuteilungen vergisst und sonst eine Adresse doppelt vergeben könnte; dann findet die Suche den c64u unter seiner neuen Adresse.

Umschalten: `setDirectMode()` speichert, verwirft den Verbindungsstatus und startet AP bzw. WLAN neu. Ein gezielt gewähltes Netz (Liste, Suche, WLAN-Karte) beendet den Direktmodus. Das Setup-Portal verdrängt den Direkt-AP vorübergehend; nach dem Portal startet `beginWiFi()` ihn wieder.

Suche im Direktmodus: Scannen kann nur der Station-Teil. Für die Dauer der Suche wird deshalb kurz auf `WIFI_AP_STA` und danach zurück auf `WIFI_AP` geschaltet.

NVS (Namensraum `c64unet`): `dmode` (bool), `dssid`, `dpass`, `dnet`. Unbrauchbare Werte (SSID leer oder > 32 Zeichen, Passwort < 8 oder > 63, Adressbereich nicht drei Zahlen 0…255) werden beim Laden durch die Vorgaben ersetzt.

Netznummer (seit v1.5.0)

Mehrere Geräte im Direktmodus an einem Tisch brauchen verschiedene Netznamen. Die Nummer steckt allein im Netznamen `gDirectSsid` (NVS `dssid`): `directNumFromSsid()` liefert 0 für `C64uRemote-Direct`, 1–9 für `C64uRemote-Direct-1` … `-9` und −1 für einen frei gesetzten Namen. `directChannel()` wählt daraus den Funkkanal für `WiFi.softAP()`: ohne Nummer wie bisher `kDirectChannel` (6), sonst reihum 1, 6, 11. Der Menüpunkt *Direkt-Nr.* (`kWifiDirectNum`) schaltet über `setDirectNum()` 0 → 1 … 9 → 0, speichert und startet einen laufenden Accesspoint neu.

Softwarearchitektur

Zustandsmodell

Der gesamte Laufzeitzustand liegt im globalen `struct AppState app`. Die Anzeige folgt einem Bildschirm-Enum:

```
enum class ScreenMode {
  Home, Menu, CpuMenu, Status, Settings, SdBrowser,
  RfidRun, RfidWrite, RfidInfo, RfidDump, RfidRestore, Busy
};
```

Gegenüber der Core-Version kommt `Menu` hinzu: Der M5Dial hat keine Kommando-Kacheln auf dem Startbild, sondern ein eigenes Ring-Menü.

Die Hauptschleife `loop()` arbeitet bei ~30 fps (`kFrameMs = 33`):

1. `M5Dial.update()` — Taste, Touch, Drehgeber aktualisieren
2. `serviceWiFi()` / `refreshConnectionStatus()` — Netzwerk pflegen
3. `handleEncoder()`, `handleButton()`, `handleTouch()` — Eingaben auswerten
4. `serviceRfid()` — RFID-Polling (alle 250 ms, nur auf RFID-Screens)
5. `updateHomeDemo()` — Animationsablauf bzw. Rückkehr zum Startbild
6. `render()` — Anzeige aufbauen und ausgeben

Rendering auf dem runden Display

Verbindungsstatus im Ring-Menü. `drawRoundFrame(withStatus, withDot)` zeichnet den Statuspunkt bei y = 14. Im Ring-Menü läge er genau unter dem Symbol *c64u Power Off*; `drawMainMenu()` ruft deshalb `drawRoundFrame(true, false)` auf und färbt stattdessen das Status-Symbol (`kMenuStatus`) mit `connectionColor()` – Zeichen immer, Rahmen nur, solange es nicht ausgewählt ist.

Der StampS3 hat kein PSRAM. Ein Vollbildpuffer bei 240 × 240 × 2 Byte belegt 115 kB und passt in den Heap; die Firmware legt ihn als `M5Canvas` an und schiebt pro Bild ein fertiges Vollbild ins Display. Das vermeidet Flackern und macht die Dirty-Flag-Logik der Core-Version überflüssig.

Schlägt `createSprite()` fehl, zeigt `gDraw` auf `M5Dial.Display` und es wird direkt gezeichnet:

```
gUseCanvas = (canvas.createSprite(kScrW, kScrH) != nullptr);
gDraw = gUseCanvas ? static_cast<lgfx::LovyanGFX*>(&canvas)
                   : static_cast<lgfx::LovyanGFX*>(&M5Dial.Display);
```

Alle Zeichenroutinen sprechen ausschließlich `gDraw` an, der Rückfall ist damit für den restlichen Code unsichtbar.

Logo 1:1. Das Logo in `1MHz_logo_rgb565.h` ist 240 × 135 Pixel groß und passt damit exakt auf die Displaybreite. Anders als in der Core-Version entfällt jede Skalierung samt Skalierungstabellen; `logoPixel()` liest direkt per `pgm_read_word` aus dem Flash.

Kreismaske. Statt für jedes Pixel den Abstand zum Mittelpunkt zu prüfen, wird pro Zeile einmal die halbe Sehnenbreite berechnet:

```
int chordHalfWidth(int y) {
  const int dy = y - kCy;
  const int inner = kRadius * kRadius - dy * dy;
  return inner <= 0 ? 0 : (int)sqrtf((float)inner);
}
```

Dieselbe Funktion begrenzt auch die Textbreite jeder Zeile, damit Beschriftungen am runden Rand nicht abgeschnitten werden.

Effekte. `drawDistortedRows` (Water/SineWave), `drawRotoZoom`, `drawRipple` und `drawRasterBars` stammen aus der Core-Version und arbeiten zeilenweise auf dem 240 × 135 großen Logobereich.

ReST-Anbindung an den C64 Ultimate

Alle Kommandos laufen über die HTTP-ReST-API der Ultimate-Firmware (ab 3.11). Basis-URL: `http://<host>/v1/…`. Ist ein Netzwerkpasswort gesetzt, wird es im Header `X-Password` mitgesendet.

Verwendete Endpunkte

Zweck	Methode	Pfad
Erreichbarkeit / Auth	GET	`/v1/version`
Kategorien auflisten	GET	`/v1/configs`
Einstellung lesen	GET	`/v1/configs/<Kategorie>/<Eintrag>`
Einstellung setzen	PUT	`/v1/configs/<Kategorie>/<Eintrag>?value=…`
Reset	PUT	`/v1/machine:reset`
Reboot	PUT	`/v1/machine:reboot`
Ultimate-Menü	PUT	`/v1/machine:menu_button`
Ausschalten	PUT	`/v1/machine:poweroff`
Speicher schreiben	PUT	`/v1/machine:writemem?address=…&data=…`
Speicher lesen	GET	`/v1/machine:readmem?address=…&length=…`
Laufwerke abfragen	GET	`/v1/drives`
Image einlegen	POST	`/v1/drives/<a|b>:mount?type=…&mode=readwrite`
Laufwerk einschalten	PUT	`/v1/drives/<a|b>:on`
Programm starten	POST	`/v1/runners:run_prg`
Modul starten	POST	`/v1/runners:run_crt`
SID abspielen	POST	`/v1/runners:sidplay`
MOD abspielen	POST	`/v1/runners:modplay`

Abgewiesene Verbindungen

Der HTTP-Server der Ultimate-Firmware nimmt jeweils nur eine Verbindung an und weist weitere mit einem TCP-RST ab; `HTTPClient` meldet das als *connection refused*. Beobachtet wurde das auch ohne ein zweites Gerät im Netz – es tritt also sporadisch auf und ist kein Funk- oder Adressproblem.

Der eigentliche Aufruf steckt in `sendApiRequestOnce()`. `sendApiRequest()` ist ein Mantel darum: Kommt keine Verbindung zustande (`HTTPC_ERROR_CONNECTION_REFUSED`), folgen nach `kApiRetryDelayMs` bis zu zwei weitere Versuche (`kApiConnectAttempts` = 3). Wiederholt wird **ausschließlich** in diesem Fall – dann ist beim c64u nichts angekommen und ein Befehl kann sich nicht doppeln. Lesefehler und HTTP-Fehlerstatus (4xx, 5xx) werden unverändert durchgereicht.

Zusätzlich spart `refreshConnectionStatus()` eine Anfrage: Ohne hinterlegtes Passwort wäre die zweite Abfrage byte-gleich mit der ersten, weil der Header `X-Password` nur gesetzt wird, wenn überhaupt eines da ist. Das halbiert die Grundlast auf dem c64u.

Wenn der c64u fehlt

Ist der c64u aus oder unter der Adresse nicht zu finden, kommt kein TCP-RST zurück, sondern gar nichts. `HTTPClient` wartet dann bis zum Ablauf seiner Verbindungsfrist, und die liegt ab Werk bei 5 s. Bis v1.3.0 stand die Hauptschleife dadurch pro Statusabfrage rund zehn Sekunden (zwei Versuche), und mit ihr Anzeige, Drehknopf und Touch.

Seit v1.3.1 gilt:

- `sendApiRequestOnce()`, `readC64Byte()` und `uploadFile()` bauen die Verbindung mit kurzer Frist auf (heute `kHttpConnectTimeoutMs` = 1,5 s je Versuch, siehe *Eigener HTTP-Weg*).
- `sendApiRequest()` wiederholt nur, wenn gar keine Verbindung zustande kam.
- Gilt der c64u als nicht erreichbar, öffnet die regelmäßige Statusabfrage mit `probeStart()` einen nicht blockierenden Socket zu Port 80 und fragt ihn in jedem Schleifendurchlauf mit `probePoll()` per `select()` ohne Wartezeit ab. Steht die Verbindung, schickt der Vortest eine vollständige Anfrage und liest die Antwort bis zum Ende. Meldet sich der c64u (Verbindung angenommen oder `ECONNREFUSED`), folgt die eigentliche Abfrage in `probeTargetNow()`; nach `kProbeConnectMs` (4,5 s) ohne Antwort bleibt es bei *Target unreachable*. Verlässt man Startbild oder Statusseite, wird der Vortest abgebrochen. Mit einem Hostnamen statt einer IP-Adresse entfällt der Vortest, weil die Namensauflösung selbst blockiert.
- Der CPU-Takt wird in `loop()` erst nachgeladen, wenn `app.connection.targetReachable` gesetzt ist.

Eigener HTTP-Weg (seit v1.4.0)

Alle Anfragen an den c64u – `sendApiRequestOnce()`, `readC64Byte()` und die Uploads in `uploadFile()` – laufen nicht mehr über `HTTPClient`/`WiFiClient`, sondern über kleine Funktionen auf lwIP-Sockets:

- `rawOpen()` baut die Verbindung nicht blockierend auf und schaut per `select()` ohne Wartezeit alle 2 ms nach; Frist `kHttpConnectTimeoutMs` (1,5 s), im Direktmodus `kHttpConnectDirectMs` (1 s).
- `rawSendAll()` schickt Anfrage bzw. Datei in Stücken mit `MSG_DONTWAIT`.
- `rawReadResponse()` liest die Antwort, bis der c64u die Verbindung schließt (oder kurz nach dem letzten per `Content-Length` angekündigten Byte), und wertet Statuszeile, `Content-Length` und `chunked` aus. Schließt der c64u ohne Antwort, lautet der Fehler *connection lost*, bleibt die Antwort aus, *read Timeout*.

`rawHttpRequest()` fasst das für GET/PUT zusammen; jede Anfrage trägt `Connection: close`. Hintergrund: Mit `HTTPClient` gingen im Heimnetz zeitweise die meisten Abfragen verloren, während eine ohne Warten abgefragte Verbindung zuverlässig durchkam.

`staSsid()` und `staRssi()` liefern Netzname und Signalstärke aus einem Zwischenspeicher, der höchstens einmal pro Sekunde über `esp_wifi_sta_get_ap_info()` erneuert wird. `WiFi.SSID()` fragt bei jedem Aufruf den Treiber, und `targetHost()` braucht den Netznamen ständig

(Erkennung des Direktnetzes als Client). `directStationCount()` erfragt die Zahl der Stationen im Direktmodus höchstens alle 500 ms.

NFC-Funkfeld

Der MFRC522-Treiber lässt das 13,56-MHz-Feld nach `PCD_Init()` dauerhaft an. `rfidFieldOn()` und `rfidFieldOff()` schalten es nur für die Kartenprobe (`cardPresent()`, `cardPresentQuick()`, danach 5 ms Anlaufzeit für die Karte) und für die Bearbeitung einer erkannten Karte ein; bei einer leeren Probe geht es wieder aus. Vor jeder Netzwerkanfrage (`rawOpen()`, `probeStart()`) wird es ebenfalls abgeschaltet – laut M5Stack teilen sich beim M5Dial RFID und WLAN die Antenne, und das WLAN ist blockiert, solange das Feld an ist. Damit eine liegen gebliebene Karte nicht bei jedem Einschalten des Feldes erneut ausgeführt bzw. beschrieben wird, merkt sich `rfidHoldCard()` nach `processCard()` ihre UID. `rfidHeldCardGone()` schaltet bei jeder Abfrage das Feld kurz ein, liest die UID und schaltet sofort wieder ab; dieselbe Karte wird ignoriert, erst zwei Fehlanzeigen in Folge oder eine andere Karte geben den Weg frei. Beim M5Dial liegt die NFC-Antenne direkt neben der WLAN-Antenne: Mit dauerhaft eingeschaltetem Feld und laufender Hintergrundabfrage gingen Pakete an den M5Dial immer wieder sekundenlang verloren, Abfragen und Uploads scheiterten.

Mesh und Roaming

`beginWiFi()` setzt `WIFI_ALL_CHANNEL_SCAN` und `WIFI_CONNECT_AP_BY_SIGNAL`: Der M5Dial sucht alle Kanäle ab und verbindet sich mit dem stärksten Zugangspunkt des Netzes statt mit dem ersten gefundenen. `serviceRoaming()` prüft im Betrieb die Signalstärke; liegt sie länger als `kRoamWeakMs` (20 s) unter `kRoamRssiDbm` (−72 dBm), startet eine Suche im Hintergrund. Ist ein anderer Zugangspunkt desselben Netzes mindestens `kRoamGainDb` (8 dB) stärker, wechselt der M5Dial per `WiFi.begin(…, Kanal, BSSID)` dorthin. Zwischen zwei Suchen liegen mindestens `kRoamGapMs` (3 min).

Netzdiagnose

Mit `kNetDiag = true` gibt die Firmware auf der seriellen Schnittstelle Zeilen mit `[NET …]` aus: jede HTTP-Anfrage mit Ergebnis und Dauer, den Vortest, WLAN-Ereignisse und alle 10 s den Netzzustand (Zugangspunkt, Kanal, RSSI, IP, ARP-Eintrag des c64u, freier Speicher). Ab Werk ist sie aus.

Wiederverbinden mit mehreren Netzen

`beginWiFi()` schaltet nach jedem Versuch auf das nächste gespeicherte Profil weiter. Ohne Gegenmaßnahme heißt das: Sind zwei Netze hinterlegt und nur eines ist erreichbar, trifft es nach einem Aussetzer jedes zweite Mal das tote Netz und kostet einen kompletten Wiederholungstakt (`kWiFiRetryMs`, 10 s).

`serviceWiFi()` merkt sich deshalb beim Verbinden über `wifiProfileIndex(staSsid())` das Profil, mit dem es geklappt hat, und legt es als nächsten Versuch fest; `gWifiNoted` sorgt dafür, dass das nur einmal je Verbindung passiert. Nach einem Aussetzer geht der erste Versuch damit wieder an das funktionierende Netz, das tote kommt nur dran, wenn das gute wirklich weg ist.

CPU-Speed

Der Pfad der Einstellung ist je nach Firmware unterschiedlich benannt. Deshalb sucht `resolveCpuPath()` zuerst gezielt in der Kategorie *U64 Specific Settings* und geht andernfalls alle Kategorien durch, bis ein Eintrag gefunden wird, dessen Name sowohl „CPU" als auch „Speed" enthält. Die Auswahlliste kommt aus dem Feld `values` der Antwort; schlägt das fehl, greift eine fest eingebaute Liste.

Joystick-Ports

Fuer das Tauschen der Joystickports gibt es keinen `machine:`-Befehl. Die Belegung ist ein Konfigurationseintrag, im Test *Joystick Swapper* in der Kategorie *U64 Specific Settings* mit

den Werten `Normal`, `Swapped`, `WASD Port 2` und `WASD Port 1`. Gesetzt wird sie deshalb ueber `/v1/configs/<Kategorie>/<Eintrag>?value=…`, genau wie die Taktstufe.

`resolveJoyPath()` sucht wie beim Takt zuerst in *U64 Specific Settings* und geht sonst alle Kategorien durch, bis ein Eintragsname „Joystick" enthaelt; ein Umbenennen durch eine spaetere Firmware faellt damit nicht auf. `refreshJoyChoices()` liest `values` und `current`.

`joyTokenFromValue()` und `joyValueFromToken()` rechnen zwischen Geraetewert und Kartenkuerzel um (`WASD Port 1` <-> `WASD1`), damit auf einer Karte kein Leerzeichen und keine firmwarespezifische Schreibweise stehen muss. `toggleJoystickSwap()` schaltet zwischen `Normal` und `Swapped` um und landet aus einem WASD-Modus wieder auf `Normal`; `cycleJoystickValue()` geht im Settings-Menue der Reihe nach durch alle gemeldeten Werte.

Disk-Images und Autostart

Beim Einlegen eines Diskettenabbilds gehören `type` und `mode` in die Query, die Datei ist der einzige Multipart-Teil. Die Ultimate-Firmware deutet jeden Multipart-Teil als Anhang; ein vorangestelltes Textfeld führt zu *Invalid type*.

Bei `Disk Action = Mnt+Run` läuft danach:

1. `PUT /v1/drives/<lw>:on` — Laufwerk einschalten bzw. zurücksetzen
2. `PUT /v1/machine:reset`
3. warten, bis der BASIC-Prompt da ist
4. `LOAD"*",8,1` in den Tastaturpuffer schreiben
5. warten, bis der Ladevorgang fertig ist
6. `RUN` in den Tastaturpuffer schreiben

Statt fester Wartezeiten wird die Adresse `$CC` (BLNSW) per `readmem` abgefragt: Ist der Wert 0, blinkt der Cursor, also wartet BASIC auf Eingaben. Getippt wird über `$0277…$0280` (Tastaturpuffer) und `$C6` (Anzahl Zeichen), vorher wird `$C5` zurückgesetzt. Weil in den Puffer nur zehn Zeichen passen, werden die PETSCII-Kurzformen `lO` und `rU` benutzt.

Streaming-Upload

`uploadFile()` öffnet eine eigene `WiFiClient`-Verbindung, schreibt die HTTP-Header von Hand und schiebt die Datei in Blöcken zu 1 kB direkt von der SD-Karte auf den Socket. Ein 175 kB großes `.d64` braucht dadurch keinen Puffer in Dateigröße – wichtig auf einem Gerät ohne PSRAM, dessen Heap zusätzlich den 115 kB großen Bildpuffer trägt.

Alle 150 ms aktualisiert `publishProgress()` den Fortschrittsbalken.

NFC-Subsystem

Zugriff auf den Leser

Der Leser wird zusammen mit dem Drehgeber initialisiert:

```
M5Dial.begin(cfg, true, true);   // Encoder, RFID
```

Angesprochen wird er über `M5Dial.Rfid` (Klasse `MFRC522`). Die Erkennung liest das Versionsregister; `0x00` und `0xFF` bedeuten „nicht ansprechbar":

```
const uint8_t version = RFID.PCD_ReadRegister(MFRC522::VersionReg);
```

Abfragestrategie

`serviceRfid()` arbeitet in zwei Betriebsarten:

1. **Auf den RFID-Seiten** (`RfidRun`, `RfidWrite`, `RfidInfo`, `RfidDump`, `RfidRestore`) wird alle 250 ms mit `cardPresent()` voll abgefragt.
2. **Auf dem Startbild und im Ring-Menue** laeuft je nach Einstellung *Auto-NFC* eine schnelle Probe mit `cardPresentQuick()`. Wird dabei eine Karte erkannt, setzt der Code

`autoRfidActive`, wechselt per `setScreen()` auf `RfidRun`, zeichnet einmal `render()` und ruft dieselbe Verarbeitung auf wie die Leseseite.

Die eigentliche Kartenbehandlung steckt in `processCard()`. Beide Pfade rufen sie auf, es gibt also nur eine Implementierung.

Warum eine eigene Probe

Liegt keine Karte auf, wartet der MFRC522 nach dem REQA-Kommando, bis sein interner Timer ablaeuft — ab Werk rund 25 ms. Genau so lange steht die Hauptschleife, was bei laufender Animation als Ruckler sichtbar waere.

Eine Karte antwortet jedoch weit schneller: die Frame Delay Time betraegt bei 106 kBit/s etwa 86 us. `cardPresentQuick()` verkuerzt das Zeitfenster deshalb nur fuer die Probe auf rund 2 ms und stellt es unmittelbar danach wieder her — noch vor `PICC_ReadCardSerial()`. Auswahl, Authentifizierung und alle Schreibvorgaenge laufen damit unveraendert mit dem vollen Zeitfenster.

```
void setRfidTimerReload(uint16_t ticks) {           // 1 Tick = 25 us
  RFID.PCD_WriteRegister(MFRC522::TReloadRegH, ticks >> 8);
  RFID.PCD_WriteRegister(MFRC522::TReloadRegL, ticks & 0xFF);
}
```

Der Ausgangswert wird nicht fest verdrahtet, sondern in `initRfid()` aus `TReloadRegH/L` gelesen und in `gRfidTimerReload` gemerkt. Aendert eine kuenftige Bibliotheksversion die Voreinstellung, bleibt das Verhalten korrekt.

Kosten. Eine ergebnislose Probe besteht aus wenigen I2C-Registerzugriffen plus den etwa 2 ms Wartezeit, zusammen grob 5 ms. Bei der Voreinstellung von 700 ms Abstand ergibt das eine Grundlast von unter einem Prozent; ein Frame von 33 ms wird dadurch nicht verfehlt.

Auto-NFC	Abstand	ungefaehre Grundlast
Off	–	0 %
1.5s	1500 ms	~0,3 %
0.7s	700 ms	~0,7 %
0.3s	300 ms	~1,7 %

Rueckkehr zum Startbild

Damit der Bildschirmschoner nach einem automatischen Start wieder uebernimmt, merkt sich `app.autoRfidActive`, dass die Leseseite nicht vom Benutzer geoeffnet wurde. Nur dann gilt fuer sie das *Home Timeout*. Navigiert der Benutzer selbst, loescht `setScreen()` das Flag.

Steht *Home Timeout* auf *Off*, greift fuer diesen Fall ersatzweise `kAutoRfidHoldMs` (20 s) - eine automatisch geoeffnete Seite bleibt also nie dauerhaft stehen.

Kartentypen

Typ	Erkennung	Ablage
MIFARE Classic 1K/4K/Mini	SAK über `PICC_GetType()`	16-Byte-Blöcke ab Block 4, Sektor-Trailer werden übersprungen
NTAG213/215/216, Ultralight	SAK 0x00	4-Byte-Seiten ab Seite 4

Bei MIFARE Classic wird zuerst mit dem NDEF-Schlüssel `D3F7D3F7D3F7` authentifiziert, ersatzweise mit dem Werksschlüssel `FFFFFFFFFFFF`. Der zuletzt erfolgreiche Schlüssel wird gemerkt, damit nicht jeder Block einen Fehlversuch kostet. Nach einem fehlgeschlagenen Auth ist die Karte im HALT-Zustand und muss über `PICC_WakeupA()` neu ausgewählt werden — das erledigt `reselectCard()`.

Kommandokarten

Traegt eine Karte statt eines Dateipfads das Praefix `CMD:`, wird der Inhalt als Befehl fuer den c64u ausgefuehrt. Weder SD-Karte noch Datei sind dafuer noetig.

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      sofort ausschalten
CMD:POWEROFF=8      nachfragen, 8 s Bestaetigungsfenster
CMD:POWEROFF        nachfragen mit der Geraeteeinstellung "NFC-Cmd PowOff"
CMD:M5OFF           den M5Dial selbst ausschalten (auch CMD:DIALOFF)
CMD:CPU=10          CPU auf 10 MHz
CMD:JOY             Joystickports umschalten (Normal <-> Swapped)
CMD:JOY=SWAPPED     Ports fest setzen; auch NORMAL, WASD1, WASD2
```

`parseCardCommand()` zerlegt den Text: Praefix pruefen, optionales Argument hinter = abtrennen, Schluesselwort in Grossbuchstaben vergleichen. Leerzeichen und Gross-/Kleinschreibung sind egal. Der Inhalt bleibt ein gewoehnlicher NDEF-Textrecord, jede NFC-App kann so eine Karte lesen und schreiben.

`processCard()` prueft im Lesezweig zuerst auf einen Befehl und ruft `runCardCommand()` auf. Traegt eine Karte zwar das Praefix, aber kein bekanntes Schluesselwort, meldet das Geraet *BEFEHL UNBEKANNT*, statt einen Dateipfad daraus zu machen.

PowerOff mit Bestaetigung

Die Wartezeit steht als Argument **auf der Karte**, nicht im Geraet; `cardPowerOffSeconds()` liefert sie zurueck. Ohne Argument gilt die Einstellung *NFC-Cmd PowOff* (3/5/8/15 s), `0` bedeutet “ohne Nachfrage”.

Der Ablauf laeuft ueber drei Felder in `AppState`:

```
bool     cardPowerOffPending;
String   cardPowerOffUid;      // nur dieselbe Karte bestaetigt
uint32_t cardPowerOffUntilMs;
```

Bestaetigt wird durch erneutes Auflegen derselben Karte oder durch einen Tastendruck. Eine andere Karte bricht ab, ebenso das Ablaufen des Fensters - beides loescht das Flag, ohne etwas auszuloesen. Die Bindung an die UID verhindert, dass eine zufaellig danebengelegte Karte den Rechner ausschaltet.

Karten beschreiben

`cmdListAt()` baut die Auswahlliste: fuenf feste Befehle, danach alle CPU-Stufen, die `app.cpuDisplayOptions` gerade enthaelt (aus dem c64u geladen, sonst die eingebaute Ersatzliste). `cardCommandText()` erzeugt daraus den Kartentext. Der Bildschirm `ScreenMode::CmdPick` zeigt die Liste, die Auswahl landet in `app.pendingCardText` und wird von der Schreibseite bevorzugt vor `pathToCardText(app.pendingPath)` verwendet.

Datenformat

Kompatibel zu TeensyROM und Zaparoo/TapTo: ein einzelner NDEF-Record vom Typ *Text* (Well Known, UTF-8, Sprachcode „en”).

```
TLV     : 03 <len> … FE
Record  : D1 01 <plen> 54 | 02 'e' 'n' | <Text>
```

Der Text ist der Pfad zur Programmdatei, zum Beispiel:

```
SD:OneLoad v5/Bubble Bobble.crt
```

Erlaubte Präfixe: `SD:`, `USB:`, `TR:` oder gar keins. `cardTextToPath()` entfernt das Präfix, fasst doppelte Schrägstriche zusammen und stellt einen führenden Schrägstrich sicher. Ein `?` als Dateiname oder ein reiner Verzeichnispfad löst über `resolveRandomFile()` eine Zufallsauswahl aus.

Zusätzlich erkennt `readCardContent()` weiterhin das ältere Rohformat mit der Kennung `C64UPATH`, damit bereits beschriebene Karten weiter funktionieren.

NDEF-Parser: Toleranz

Manche Schreibprogramme tragen eine falsche Payload-Länge ein — der TeensyROM zum Beispiel konstant `0x10`. Beim letzten Record (ME-Flag gesetzt) hat deshalb die aus der TLV-Länge abgeleitete Größe Vorrang. Zusätzlich filtert `sanitizeCardText()` alle Zeichen unterhalb 0x20 heraus, weil manche Apps NUL, CR oder LF anhängen und damit jede Dateiendung unbrauchbar machen würden.

NFC-Info

`collectCardInfo()` sammelt UID, SAK, Typ, Speichergröße, das Ergebnis von `GET_VERSION` (0x60) sowie den Karteninhalt. `collectCardRaw()` ergänzt Rohdaten: bei NTAG die Seiten 0–15, Lock-Bytes, Capability Container, den Passwortschutz aus der Konfigurationsseite und den Lesezähler (`READ_CNT`, 0x39); bei Classic die Blöcke 0, 4, 5, 6 und 8 sowie den verwendeten Schlüssel.

`buildInfoView()` bricht das Ergebnis auf 30 Zeichen pro Zeile um und legt es in einer scrollbaren Liste ab — auf einem 240 × 240 großen Display passen elf Zeilen gleichzeitig.

Befehle, die eine Karte abmelden können, stehen bewusst am Ende: Schlägt `GET_VERSION` fehl, folgt sofort ein `reselectCard()`.

Kopieren (Dump/Restore)

`dumpCardToSd()` schreibt eine Textdatei nach `/NFC-DUMPS/<UID>.nfc`:

```
# C64uRemote NFC-Dump
type NTAG215
uid  04 01 A1 01 C1 47 03
sak  00
P4   03 27 D1 01
```

Bei MIFARE Classic wird pro Sektor ein Schlüsselwörterbuch mit 13 gängigen Schlüsseln durchprobiert (`classicAuthDict()`), sodass sich auch fremde Karten sichern lassen. Da Schlüssel A nie lesbar ist, trägt der Dump den tatsächlich gefundenen Schlüssel in den Trailer ein.

`restoreDumpToCard()` schreibt nur Nutzdaten zurück:

- NTAG/Ultralight ab Seite 4 bis zur letzten Nutzseite; Seiten 0–3 (UID, Lock, Capability Container) und die Konfigurationsseiten bleiben tabu
- Classic ohne Block 0; Sektor-Trailer nur, wenn `validAccessBits()` das Zugriffsbitmuster als stimmig bestätigt

Beide Einschränkungen verhindern, dass eine Karte dauerhaft unbrauchbar wird. Die UID selbst ist fest im Chip und wird nicht kopiert.

Bedienlogik

Drehgeber

`M5Dial.Encoder.read()` liefert sehr feine Tickfolgen. `handleEncoder()` sammelt sie in `app.encoderResidual` und gibt erst nach der eingestellten Anzahl Ticks (*Encoder Steps*: 1, 2, 4 oder 6) einen Menüschritt weiter. Auf dem Startbild öffnet die erste Drehung nur das Menü, ohne die Auswahl zu verschieben.

Taste

Der M5Dial hat nur eine Taste, deshalb entfallen die Tastenkombinationen der Core-Version:

Ereignis	Wirkung
kurzer Druck	`handleSelect()`
langer Druck (≥ 600 ms) auf dem Startbild	`runShortcut(shortcutButton)`
langer Druck sonst	`handleBack()`

Das Flag `app.buttonHandled` sorgt dafür, dass nach einem ausgewerteten langen Druck das Loslassen keine zweite Aktion mehr auslöst.

Touch

`handleTouch()` merkt sich Position und Zeitpunkt beim ersten erkannten Kontakt und wertet beim Loslassen aus. Auf dem Startbild und im Ring-Menü löst ein Halten ab 600 ms `runShortcut(shortcutTouch)` aus; der Tipper beim Loslassen entfällt dann.

Die Trefferzuordnung im Ring-Menü läuft zweistufig (`menuIndexFromTouch()`):

1. direkter Treffer im Umkreis von `kMenuHitRadius` = 30 px um ein Symbol
2. sonst über `atan2()` der Winkel zur Mitte, aufgeteilt in zehn Sektoren zu 36°

Damit ist der gesamte Ring außerhalb von `kMenuRingInner` = 50 px aktiv. Innerhalb von `kMenuCenterRadius` = 50 px liegt das Logo, ein Tipper dort führt zurück zum Startbild. In Listen zählt der Zwischenraum zwischen zwei Zeilen zur oberen Zeile, damit es keine wirkungslosen Streifen gibt.

Blockierende Aufrufe und Eingaben

Der zyklische Verbindungstest kostet zwei HTTP-Aufrufe mit je 3 s Timeout und blockiert die Hauptschleife entsprechend; währenddessen wird der Touchscreen nicht abgefragt. `refreshConnectionStatus()` führt den zyklischen Test deshalb nur auf dem Startbild und auf der Statusseite aus. Während der Bedienung läuft er ausschließlich auf ausdrückliche Anforderung (`force = true`), also beim Öffnen der Statusseite und bei einem Druck auf ihr.

PowerOff-Absicherung

Zwei getrennte Mechanismen:

- **Menüpunkt c64u Power Off:** `requestPowerOff()` setzt `pendingPowerOff`; erst ein zweiter Druck innerhalb von *PowerOff Zeit* schaltet ab. Das gilt immer.
- **Tastenkürzel PowerOff:** je nach *PowerOff Abfrage* entweder sofort oder mit Rückfrage (`comboPowerOff`), die mit einem weiteren Druck bestätigt wird.

Der M5Dial selbst hat eine eigene Absicherung, siehe *Akkubetrieb und Ausschalten*: zweimal drücken in den Einstellungen, 1,5 s halten im Ring-Menü.

Persistenz

Einstellungen liegen im NVS unter dem Namensraum `c64udial` (die Core- und StickC-Fassungen benutzen `c64uremote`, es gibt also keine Kollision). Gespeichert werden Anzeige- und Bedienoptionen, Disk-Optionen, die beiden Tastenkürzel und die PowerOff-Parameter. Beim Laden wird jeder Wert gegen seinen gültigen Bereich geprüft; unplausible Werte fallen auf die Vorgabe zurück.

Die WLAN-Zugangsdaten liegen in einem **eigenen** Namensraum `c64unet` (siehe Kapitel *WLAN-Subsystem*) und bleiben deshalb auch nach *Factory Reset* erhalten. `build_env.h` liefert nur noch die Startwerte, solange dort nichts gespeichert ist.

Projektstruktur

```
M5Dial-C64uRemote/
```

```
├── platformio.ini            Board, Bibliotheken, Partition, Upload
├── README.md
├── LICENSE                   MIT - Karl Prosser, Martin Oswald
├── wifi.txt.example          Vorlage für /wifi.txt auf der SD-Karte
├── .vscode/                  empfohlene Erweiterungen, Editor-Einstellungen
├── docs-src/                 Markdown-Quellen der Handbücher
├── doc/                      fertige Handbücher (PDF)
├── src/                      deutsche Fassung
│   ├── main.cpp              gesamte Firmware
│   ├── build_env.h           Zugangsdaten (nicht versionieren)
│   ├── build_env.h.example   Vorlage
│   └── 1MHz_logo_rgb565.h    Logo, 240 × 135, RGB565, PROGMEM
└── src-en/
    └── main.cpp              englische Fassung, Code identisch
```

Fehlerdiagnose

Symptom	Ursache / Abhilfe
SET build_env.h beim Start	Zugangsdaten fehlen
KEIN NFC-LESER	Serielle Ausgabe prüfen: `RFID (I2C 0x28) VersionReg = …. 0x00/0xFF` heißt kein Kontakt zum internen Bus
KEINE SD-KARTE	Verdrahtung, Versorgungsspannung des Moduls, FAT32, Kabellänge
KEINE SD-KARTE nach Aufbau laut Doku bis v1.2.1	An Port B gelbe und weiße Ader tauschen: gelb = MISO (G2), weiß = CS (G1)
Statuspunkt gelb (*AUTH?*)	`C64U_TARGET_PASSWORD` stimmt nicht
Invalid type beim Mounten	`type/mode` gehören in die Query, nicht als Multipart-Textfeld
LADEN ZU LANG	Der Ladevorgang überschreitet 180 s; Image prüfen oder *Disk Action* auf *Mnt+Reset* stellen
Karte lesbar, Datei fehlt	Pfad auf der Karte passt nicht zur SD; *NFC-Info* zeigt es an
Anzeige ruckelt	*Effect* auf `Static` oder *Effect Time* verkürzen
`Kein Speicher fuer den Offscreen-Puffer` im Log	Heap knapp; Firmware läuft weiter, zeichnet aber direkt ins Display

Quellen

- ReST-API des C64 Ultimate: `https://1541u-documentation.readthedocs.io`
- Originalprojekt: `https://github.com/ReadyOS-C64/C64uRemote`
- M5Dial: `https://docs.m5stack.com/en/core/M5Dial`
- M5Dial-Bibliothek: `https://github.com/m5stack/M5Dial`
- Kartenformat: TeensyROM NFC-Loader, Zaparoo/TapTo

Lizenz

C64uRemote steht unter der **MIT-Lizenz**. Der vollständige Lizenztext liegt als Datei `LICENSE` im Projektstamm.

Ursprung ist das Projekt **C64uRemote von Karl Prosser (@klumsy)**, https://github.com/ReadyOS-C64/C64uRemote, das er unter der MIT-Lizenz veröffentlicht hat. Diese Fassung ist eine daraus abgeleitete Erweiterung und steht unter denselben Bedingungen:

- Copyright (c) 2026 Karl Prosser – Originalprojekt

- Copyright (c) 2026 Martin Oswald (@mad, https://1MHz.de) – Portierung und Erweiterungen

Die MIT-Lizenz erlaubt es, die Software zu benutzen, zu verändern und weiterzugeben, auch kommerziell. Einzige Bedingung: **Copyright-Vermerk und Lizenztext müssen erhalten bleiben** und jeder Kopie beiliegen. Eine Gewährleistung oder Haftung ist ausgeschlossen.

Die eingebundenen Bibliotheken haben ihre eigenen Lizenzen: M5Unified und M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) sowie MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). Sie werden beim Bauen von PlatformIO geladen.

Entstehung

Portierung, Erweiterungen und Handbücher sind mit Unterstützung von Claude (Anthropic) entstanden. Konzept, Idee, Hardware-Entscheidungen und sämtliche Tests auf den echten Geräten: Martin Oswald (@mad).